MODULE 34 · DOMAIN 6 - EVALUATION AND MONITORING

Monitoring and cost control in production

Inference tables, usage tracking and the AI Gateway: seeing what a live application does, and keeping its costs in check.

SECTION 6 of 6 EXAM WEIGHT 12% MODULE 34 of 40

admin@sthithapragna.com

A meter, a speed limiter and CCTV. Usage tracking counts what is consumed, rate limits cap it, and inference tables record what was actually said.

What the gateway adds to an endpoint

FEATURE	WHAT IT DOES	COST
Permissions and rate limits	Who may call, and how much, per user or group	Free
Usage tracking	Requests and tokens by requester, into system tables	Paid
Payload logging	Requests and responses, into inference tables	Paid
Traffic splitting, fallbacks	Route between models, and fail over for external models	Free
Guardrails	Checks on input and output, from module 17	Preview

ON THE EXAM

The gateway is the single place to govern, meter and record model traffic, whichever model sits behind it.

What gets recorded, and how to use it

ASPECT	DETAIL
Contents	Request and response payloads, status and requester, per call
Where	A Delta table in Unity Catalog, queryable with SQL
Timing	Rows usually appear within an hour
Limits	Payloads over 1 MiB are not logged
Versions	Legacy inference tables now apply only to custom model endpoints

ON THE EXAM

Inference tables are the raw record: use them to debug, audit, and seed evaluation datasets from real traffic.

Usage tracking, or inference tables?

USAGE TRACKING

Who, and how much

- Request and token counts
- By user or service principal
- system.serving.endpoint_usage
- No prompt or response text

INFERENCE TABLES

What was said

- Full request and response payloads
- Debugging and audit
- Seed evaluation datasets
- Heavier, and can hold sensitive text

THE TIEBREAKER

Counting consumption by user is usage tracking. Reading what was asked and answered is inference tables.

Inference tables, or MLflow monitoring?

INFERENCE TABLES

The raw record

- Every request and response
- Analysed with SQL
- Audit and investigation
- No quality scores by themselves

MLFLOW PRODUCTION MONITORING

Quality on traces

- Structured traces with spans
- Judges and scorers on a sample
- Quality trends and alerts
- From module 20

THE TIEBREAKER

To know whether answers are good, score traces with judges. To know exactly what was said, query the inference table.

From logs to a better release

01
Log
Payload logging on, usage tracking on.

02
Find
Query for errors, complaints, empty answers.

03
Sample
Add failing requests to the evaluation set.

04
Score
Run judges; locate the cause in traces.

05
Fix
Release through the gate from module 27.

ON THE EXAM

Inference logs turn real failures into test cases. That is how the evaluation set keeps up with users.

admin@sthithapragna.com

The cost levers from across the course

LEVER	FROM
The smallest model that passes evaluation	Modules 11 and 33
Pay-per-token for spiky use; no idle target QPS	Modules 22 and 24
Batch with ai_query instead of per-row calls	Module 24
Triggered index sync, not continuous	Module 21
Fewer, smaller chunks and a max_tokens cap	Modules 08 and 24
Per-user rate limits	Module 29

ON THE EXAM

Most cost control is design, decided long before the bill arrives. Monitoring tells you which lever to pull.

Where cost shows up on Databricks

- **system.billing.usage** holds the cost of model serving.
- **Custom tags** on endpoints, such as team, flow into its custom_tags column.
- The gateway adds endpoint, destination model and requester to billing records. In Beta.
- **Budgets** send alerts at thresholds you set, up to four per budget.
- Budgets are not a hard cap, and alerts can lag by up to 24 hours.

> ON THE EXAM
>
> Databricks recommends tagging every serving endpoint, so cost can be attributed by team or use case.

A budget alert, or a rate limit?

BUDGET ALERT

Tells you

- Email at a spending threshold
- Can arrive up to a day later
- Not a hard cap on spend
- Good for oversight

RATE LIMIT

Stops it

- Caps requests or tokens per minute
- Applies immediately
- Returns 429 above the limit
- Good for enforcement

THE TIEBREAKER

If the requirement is that usage must never exceed a limit, you need a rate limit. A budget only tells you afterwards.

Harbourline's production dashboard

01
Meter
Usage tracking by team and user.

02
Limiter
Per-user rate limits on customer chat.

03
CCTV
Inference tables, masked, for audit.

04
Quality
MLflow judges on sampled traces.

05
Cost
Tagged endpoints, budget alerts to finance.

ON THE EXAM

Each question has its own instrument: consumption, enforcement, record, quality and cost.

> An engineering lead wants to know which users consumed the most tokens on a chat endpoint yesterday.
>
> Usage tracking and payload logging are both enabled on the endpoint.

Where should the lead look?

A Read the inference table payloads and count the words

B Query the usage tracking system table by requester

C Read the MLflow judge scores on production traces

D Check the endpoint's replica count in the serving UI

◆ PAUSE AND DECIDE

Where should the lead look?

A Count words in payloads

Words are not tokens, and it reads sensitive text needlessly.

B Usage tracking by requester

Token counts by user are exactly what it records.

C Judge scores

Quality, not consumption.

D Replica count

Capacity, not who used it.

THE SIGNAL

Who consumed how many tokens is a metering question. The meter is usage tracking.

> A company runs forty serving endpoints for many teams. Finance wants model serving cost broken down by team each month.
>
> Today the bill shows only a single total.

What should the platform team do?

A. Ask each team to estimate its share of the bill monthly

B. Move every team's traffic onto a single shared endpoint

C. Read the inference table payloads and count the tokens

D. Tag each endpoint with its team, then query the billing table

◆ PAUSE AND DECIDE

What should the platform team do?

A Team estimates

Guesswork, not attribution.

B One shared endpoint

Makes attribution harder, not easier.

C Count tokens in payloads

Tokens are not cost, and it misses provisioned capacity.

D Tag endpoints, query billing

Tags flow into custom_tags in system.billing.usage.

THE SIGNAL

Cost by team. Tag every endpoint, and the billing system table does the attribution. Databricks recommends exactly this.

> A pilot allows each user a fixed number of requests per minute. Last week one user exceeded it for hours.
>
> The team's monthly budget alert only emailed finance the following day.

What should the team change?

A. Set per-user rate limits on the gateway endpoint

B. Lower the budget threshold so the alert fires earlier

C. Add a fourth alert to the existing monthly budget

D. Turn on payload logging to record every request

◆ PAUSE AND DECIDE

What should the team change?

A Per-user rate limits

Enforced immediately. Excess requests get 429s.

B Lower budget threshold

Still an alert, still delayed, still not a cap.

C A fourth budget alert

More warnings, no enforcement.

D Payload logging

Records the overuse. Does not stop it.

THE SIGNAL

A fixed limit per user that must hold. Budgets tell you afterwards; rate limits stop it at once.

> Complaints about a deployed rag assistant have risen. Payload logging is enabled on its endpoint.
>
> The team wants to find the failing requests and turn them into test cases for the next release.

What should the team do?

A Look only at the endpoint's average latency chart

B Raise the rate limits so more users can reach the app

C Query the inference table and add failures to the eval set

D Move the endpoint from pay-per-token to provisioned mode

◆ PAUSE AND DECIDE

What should the team do?

A Average latency chart

Speed says nothing about which answers were wrong.

B Raise rate limits

More traffic to a failing app.

C Query logs, sample into evals

Finds real failures and makes them test cases.

D Provisioned mode

Capacity, not correctness.

THE SIGNAL

Find failing requests, then make them test cases. The inference table is where real requests and answers live.

What to carry forward

- Meter: **usage tracking**. Limiter: **rate limits**. CCTV: **inference tables**.
- Inference tables: payloads in Unity Catalog, within an hour, not over 1 MiB.
- Quality on traces with **MLflow monitoring**. Raw record in **inference tables**.
- Cost: **tag endpoints**, query system.billing.usage, set budgets for oversight.
- Budgets alert late and do not cap. Rate limits enforce immediately.

> ON THE EXAM
>
> That completes the teaching modules. Next: module 35 collects everything that has changed since the exam guide.